

Entrega Laboratorio 5

Network Segmentation pattern

Integrantes Grupo 2D:

Stiven Aguirre Granada staguirreg@unal.edu.co
Juan Jose Alvarez Lozano Jualvarezlo@unal.edu.co
David Andrés Camelo Suárez dcamelos@unal.edu.co
Juan Manuel Torres Leon jutorresle@unal.edu.co
Sergio Alejandro Reita Serrano sreita@unal.edu.co
Maria Paula Román Arévalo maromana@unal.edu.co
Julian David Rodriguez Fernandez jrodriguezfe@unal.edu.co
John Jairo Paez albino jopaeza@unal.edu.co

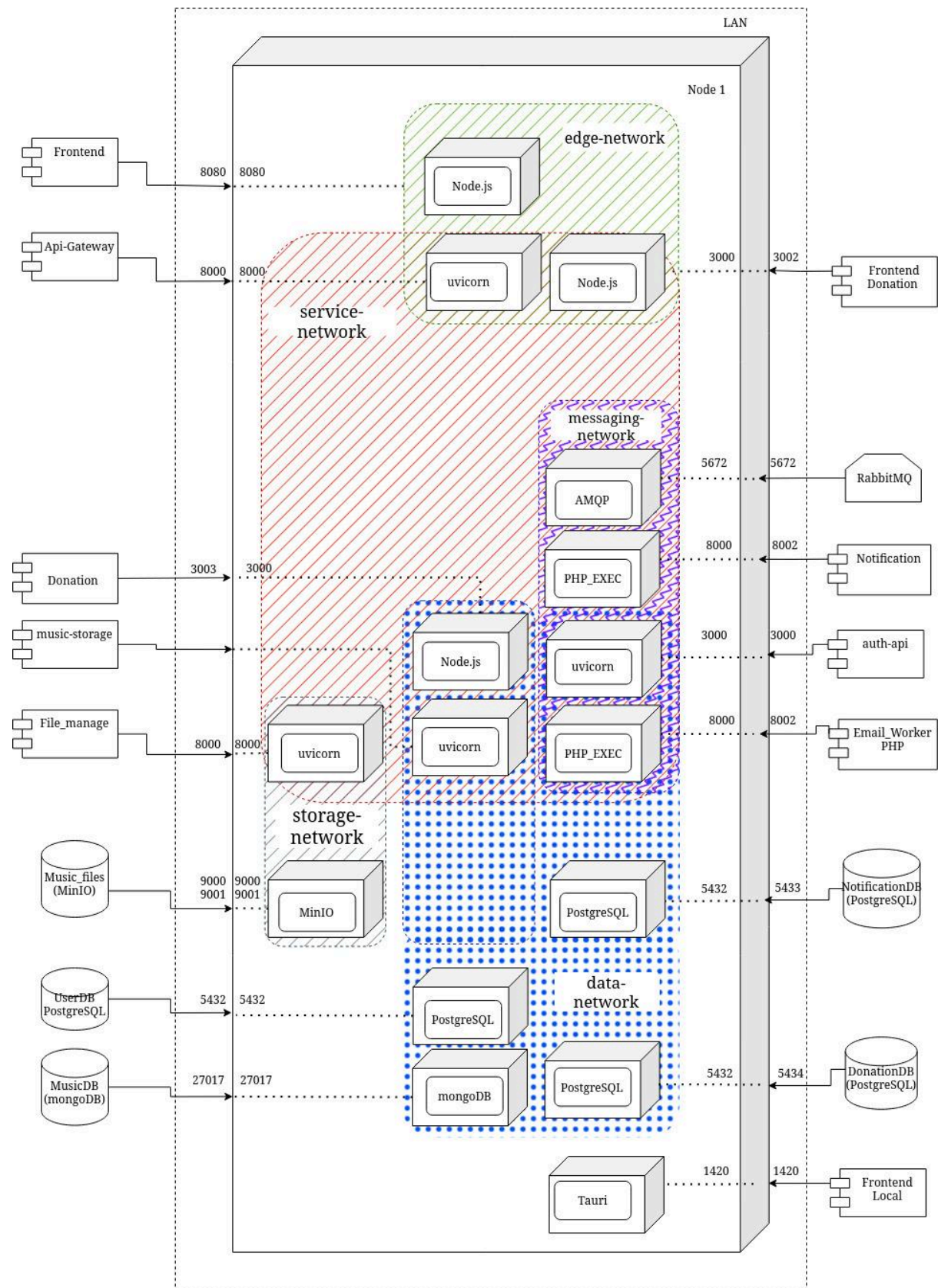
Ingeniería de Sistemas y Computación
Arquitectura de Software



UNIVERSIDAD
NACIONAL
DE COLOMBIA

11 de Mayo del 2026

Vista Arquitectónica



Guía Técnica

Descripción del Patrón

El patrón de Segmentación de Red(Network Segmentation Pattern) divide una red de computadoras en subredes más pequeñas. Su propósito principal es mejorar la seguridad y el rendimiento aislando los servicios y limitando la comunicación únicamente a las rutas necesarias. De esta manera, si un componente es comprometido, el atacante no puede moverse lateralmente con facilidad hacia otras partes del sistema.

Escenarios dirigidos a la calidad

- **Seguridad (Confidencialidad e Integridad):**

Al segmentar la red, se evita que servicios expuestos públicamente (como el frontend o el API Gateway) tengan acceso directo a las bases de datos o a la cola de mensajería. Esto garantiza que la información sensible (**confidencialidad**) y el estado del sistema (**integridad**) estén protegidos frente a accesos no autorizados provenientes de la capa más externa de la aplicación.

- **Disponibilidad:**

La segmentación de red fortalece la **disponibilidad** al reducir el radio de explosión de cualquier incidente, limitando fallos técnicos, congestiones de tráfico y ataques de malware a un solo sector para evitar caídas sistémicas. Este aislamiento lógico permite aplicar políticas de calidad de servicio (**QoS**) y realizar mantenimientos controlados, garantizando que los servicios críticos permanezcan operativos y estables incluso si un segmento secundario se ve comprometido.

Pasos seguidos para la implementación en el proyecto

- **Paso 1: Identificar dominios de comunicación**

Categoría	Servicios
Exposición pública	frontend, api-gateway
Microservicios	user-api, files-api, etc
Bases de datos	postgres, mongo
Mensajería	rabbitmq
Almacenamiento	minio

- Paso 2: Crear redes específicas

None

```
networks:
  edge-network:
    driver: bridge

  services-network:
    driver: bridge

  data-network:
    driver: bridge

  messaging-network:
    driver: bridge

  storage-network:
    driver: bridge
```

Cada dominio obtiene su propia red.

- Paso 3: Conectar servicios solo donde era necesario

Se reemplazó *'main-network'* en cada servicio por las redes específicas que requiere. Por ejemplo, *'api-gateway'* se conecta a *'edge-network'* y *'services-network'*, actuando como puente. Las bases de datos solo se conectan a *'data-network'*.

Ejemplo:

None

```
metadata-api:
  networks:
    - services-network
    - data-network
```

Esto permite:

- Acceder a MongoDB
- Comunicarse con otros microservicios

Pero **NO** acceder directamente al frontend.

- Paso 4: Restringir redes internas

Uso de:

```
None  
internal: true
```

Para impedir acceso desde la red pública.

Por ejemplo:

```
None  
networks:  
  
  services-network:  
    driver: bridge  
    internal: true  
  
  data-network:  
    driver: bridge  
    internal: true  
  
  storage-network:  
    driver: bridge  
    internal: true
```

Configuración y código utilizado

Redes del Docker-compose.yml original del prototipo 2:

```
None  
networks:  
  main-network:  
    driver: bridge
```

Una red única para todos los contenedores, dando riesgo a fallos en cadena, además de representar un punto débil de seguridad.

Nuevas redes implementadas:

None

postgres:

- networks:
 - data-network

user-api:

- networks:
 - services-network
 - data-network
 - messaging-network

payments-postgres:

- networks:
 - data-network

payments_app:

- networks:
 - services-network
 - data-network

mf_payments:

- networks:
 - services-network
 - edge-network

mongo:

- networks:
 - data-network

minio:

- networks:
 - edge-network
 - storage-network

metadata-api:

- networks:
 - services-network
 - data-network

files-api:

- networks:
 - services-network
 - data-network
 - storage-network

api-gateway:

- networks:
 - edge-network

```

- services-network

frontend:
  networks:
    - edge-network

notification-rabbitmq:
  networks:
    - messaging-network

notification-postgres:
  networks:
    - data-network

notification-producer:
  networks:
    - services-network
    - messaging-network

notification-worker:
  networks:
    - services-network
    - data-network
    - messaging-network

```

Redes dedicadas con tráfico separado, separando la lógica y responsabilidad de cada tipo de tráfico. También se reduce el radio de explosión de cualquier incidente, limitan fallos técnicos, congestiones de tráfico y ataques.

Resultados y mejoras observadas

- **Aislamiento efectivo:**
Las bases de datos (*postgres*, *mongo*, *minio*) ya no son accesibles desde contenedores en la red *edge-network*.
- **Reducción de la superficie de ataque:**
Un atacante que logre vulnerar el frontend no podrá escanear ni conectarse directamente a los puertos de la base de datos.
- **Trazabilidad:**
Es más fácil monitorear el tráfico de red, ya que el propósito de cada subred está claramente definido.

Recomendaciones para replicar la implementación

- **Principio de Mínimo Privilegio:**
Asegúrese de que cada contenedor solo tenga acceso a las redes estrictamente necesarias.

- **Validar la conectividad:**

Al segmentar, es común romper la comunicación legítima entre servicios. Prueba exhaustivamente los flujos completos de la aplicación tras aplicar los cambios.

- **Uso de `'internal: true'`:**

Para todas las redes que no requieran exposición externa (bases de datos, mensajería), siempre utilice esta bandera en Docker Compose para aislar el tráfico interno.

Anexo:

Docker-compose.yml implementado:

```
Shell
services:
  # CAPA DE DATOS

  postgres:
    image: postgres:15-alpine
    container_name: proyectosoft-postgres
    environment:
      POSTGRES_USER: authuser
      POSTGRES_PASSWORD: authpass
      POSTGRES_DB: auth_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U authuser -d auth_db"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks:
      - data-network

  payments-postgres:
    image: postgres:15-alpine
    container_name: payments-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: payments_db
    volumes:
      - payments_postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d payments_db"]
      interval: 10s
      timeout: 5s
```



```

    retries: 5
networks:
  - data-network

mongo:
  image: mongo:7
  container_name: proyectosoft-mongo
  volumes:
    - mongo_data:/data/db
  networks:
    - data-network

notification-postgres:
  image: postgres:15-alpine
  container_name: notification-postgres
  environment:
    POSTGRES_DB: emails_db
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
  volumes:
    - postgres_notification_data:/var/lib/postgresql/data
    -
./Notification/docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres -d emails_db"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - data-network

# CAPA DE APLICACIÓN

user-api:
  build:
    context: ./auth-api
  container_name: proyectosoft-user-api
  environment:
    DATABASE_URL: postgres://authuser:authpass@postgres:5432/auth_db
    JWT_SECRET:
${JWT_SECRET:-your-256-bit-secret-key-here-change-in-production}
    NOTIFICATION_URL: http://notification-producer:8000
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - services-network
    - data-network

```

```
payments_app:
  build:
    context: ./payments_app
    dockerfile: Dockerfile
  container_name: payments_app
  environment:
    NODE_ENV: development
    PORT: 3000
    DB_HOST: payments-postgres
    DB_PORT: 5432
    DB_NAME: payments_db
    DB_USER: postgres
    DB_PASSWORD: postgres
    FRONTEND_URL: http://localhost:8080
  depends_on:
    payments-postgres:
      condition: service_healthy
  volumes:
    - ./payments_app:/app
    - /app/node_modules
  command: npm run dev
  networks:
    - services-network
    - data-network

mf_payments:
  build:
    context: ./mf_payments
    dockerfile: Dockerfile
  container_name: mf-payments-frontend
  environment:
    NODE_ENV: development
    PAYMENTS_API_URL: http://payments_app:3000
  ports:
    - "3003:3000"
  depends_on:
    - payments_app
  networks:
    - edge-network
    - services-network

metadata-api:
  build:
    context: ./metadata-api
  container_name: gateway-metadata-api
  environment:
    MONGO_URI: mongodb://proyectosoft-mongo:27017
```

```
    DB_NAME: music_db
    JWT_SECRET:
${JWT_SECRET:-your-256-bit-secret-key-here-change-in-production}
    JWT_ALGORITHM: HS256
    BUCKET_NAME: scores
    MINIO_PUBLIC_URL: http://localhost:9000
depends_on:
  mongo:
    condition: service_started
networks:
  - services-network
  - data-network

files-api:
  build:
    context: ./files-api
  container_name: gateway-files-api
  environment:
    MINIO_ENDPOINT: proyectosoftware-minio:9000
    MINIO_ROOT_USER: admin
    MINIO_ROOT_PASSWORD: password
    BUCKET_NAME: scores
  depends_on:
    minio:
      condition: service_started
  networks:
    - services-network
    - data-network
    - storage-network

notification-producer:
  build:
    context: ./Notification
    dockerfile: Dockerfile
  container_name: notification-producer
  command: ["php", "-S", "0.0.0.0:8000", "router.php"]
  depends_on:
    notification-rabbitmq:
      condition: service_healthy
  environment:
    RABBITMQ_HOST: notification-rabbitmq
    RABBITMQ_PORT: 5672
  networks:
    - services-network
    - messaging-network

notification-worker:
  build:
```

```

    context: ./Notification
    dockerfile: Dockerfile
    container_name: notification-worker
    command: ["php", "email.php"]
    depends_on:
      notification-rabbitmq:
        condition: service_healthy
      notification-postgres:
        condition: service_healthy
    environment:
      RABBITMQ_HOST: notification-rabbitmq
      RABBITMQ_PORT: 5672
      SENDGRID_API_KEY: ${SENDGRID_API_KEY:-}
      DB_HOST: notification-postgres
      DB_PORT: 5432
      DB_NAME: emails_db
      DB_USER: postgres
      DB_PASSWORD: password
    networks:
      - services-network
      - data-network
      - messaging-network
    restart: unless-stopped

notification-rabbitmq:
  image: rabbitmq:3-management-alpine
  container_name: notification-rabbitmq
  environment:
    RABBITMQ_DEFAULT_USER: guest
    RABBITMQ_DEFAULT_PASS: guest
  ports:
    - "15672:15672" # Expón solo la UI de gestión para acceso web (no el
puerto 5672 para prod)
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - messaging-network

# INFRAESTRUCTURA DE ALMACENAMIENTO

minio:
  image: minio/minio
  platform: linux/amd64
  container_name: proyectosoft-minio
  environment:

```

```
MINIO_ROOT_USER: admin
MINIO_ROOT_PASSWORD: password
command: server /data --console-address ":9001"
ports:
  - "9000:9000" # API pública si lo requieres.
  - "9001:9001" # UI de administración
volumes:
  - minio_data:/data
networks:
  - storage-network
  - edge-network
```

CAPA DE EXPOSICIÓN

```
api-gateway:
  build:
    context: ./api-gateway
  container_name: gateway-api-gateway
  environment:
    DEBUG: "True"
    GATEWAY_PORT: 8000
    USER_API_URL: http://user-api:3000
    NOTIFICATION_URL: http://notification-producer:8000
    METADATA_API_URL: http://metadata-api:8000/storage
    FILES_API_URL: http://files-api:8000
    FRONTEND_URL: http://localhost:8080
  ports:
    - "8000:8000"
  depends_on:
    user-api:
      condition: service_started
    metadata-api:
      condition: service_started
    files-api:
      condition: service_started
    notification-producer:
      condition: service_started
  networks:
    - edge-network
    - services-network

frontend:
  build:
    context: ./Front-end
  container_name: proyectosoft-frontend
  environment:
    VITE_API_URL: http://localhost:8000
```

```
ports:
  - "8080:8080"
depends_on:
  api-gateway:
    condition: service_started
networks:
  - edge-network

volumes:
  postgres_data:
  payments_postgres_data:
  mongo_data:
  minio_data:
  postgres_notification_data:

networks:
  edge-network:
    driver: bridge
  services-network:
    driver: bridge
    internal: true
  data-network:
    driver: bridge
    internal: true
  messaging-network:
    driver: bridge
  storage-network:
    driver: bridge
    internal: true
```